Document 20 — Web Frontends for AI Apps (Streamlit, Flask, FastAPI, Django)

> This is for self-study and live mentoring. It teaches how to put a **web interface** on the AI pipelines you built in Projects 1-20 — so a real person can use them in a browser, not just a terminal.

Prerequisites

12_production_engineering (FastAPI basics) and any project whose pipeline you want to serve — e.g. Project 14 InvoiceIQ or [Project 17's chatbot]. You only need one Python function to call (`run_pipeline(text) -> result`); this document is about wrapping it.

How to Read & Practice This Document

- **What:** serving an AI function over the web with four tools — **Streamlit** (fastest), **Flask** (minimal), **FastAPI** (async APIs, from Doc12), and **Django** (batteries-included).
- **Why:** a CLI script impresses no one; a link someone can click does. Every one of these projects becomes a real, shareable app with ~30 lines of web code.
- **When:** the moment a human other than you needs to use your pipeline. Pick the lightest tool that fits — Streamlit for a demo, Flask for a small app, FastAPI for an API, Django when you need users/admin/a database.
- **How to practice:** read the walkthrough, then build the **same** tiny classifier UI in each framework. They all do the same thing; the differences are the whole lesson.

Jump to: Learn It in Order (start here) · Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Every project so far ends at a terminal: you run `python main.py` and read the output. That's fine for you, useless for anyone else. This document is the last mile — the web layer that turns `run_pipeline(text)` into a page someone opens in a browser, types into, and gets an answer from.

The pattern is the **same in all four tools**: a page (or an endpoint) takes the user's input, calls your pipeline function, and shows the result. **Streamlit** writes that page for you in pure Python — no

HTML, no routes — so it's the fastest way to a demo. **Flask** is a tiny framework where you write the route and a small HTML template yourself — more control, still minimal. **FastAPI** (which you met in Doc12) is built for JSON APIs and async, and can serve a form too. **Django** is the heavyweight: it comes with users, an admin panel, and a database ORM out of the box, for when the app is more than one page. Learn the one shared pattern once, and you can reach for whichever tool the job actually needs.

Learn It in Order — Serve One Function, Four Ways

New to web apps? Start here. We take **one** tiny function — a sentiment classifier — and put a web interface on it four times. The function never changes; only the wrapper does. Type each one and run it. Then the Core Concepts below explain each framework more deeply.

> Setup: `pip install streamlit flask fastapi "uvicorn[standard]" django jinja2`. In real projects `classify` is your agent or pipeline; here it's a stub so the focus stays on the web layer.

Step 0 — The function (this is what we're serving)

```
# pipeline.py — in a real project this calls your agent/RAG/LCEL pipeline
def classify(text: str) -> str:
    lowered = text.lower()
    for word in ["love", "great", "amazing", "good"]:
        if word in lowered:
            return "positive"
    for word in ["bad", "hate", "terrible", "awful"]:
        if word in lowered:
            return "negative"
    return "neutral"
```

Every framework below calls exactly this `classify(text)`. Nothing about it is web-specific — that's the point.

Step 1 — Streamlit: a UI in pure Python (fastest)

```
# app_streamlit.py   —   run with:  streamlit run app_streamlit.py
import streamlit as st

from pipeline import classify

st.title("Sentiment Classifier")
```

```
text = st.text_input("Type something:")
if st.button("Classify") and text:
    st.write("Result:", classify(text))
```

What's new: no HTML, no routes, no server code — Streamlit turns a Python script into a live web page, re-running it top to bottom on every interaction. Best for a demo or an internal tool you need *today*.

Step 2 — Flask: write the route and a small template

```
# app_flask.py   —   run with:  flask --app app_flask run
from flask import Flask, request, render_template_string

from pipeline import classify

app = Flask(__name__)
PAGE = """
<form method="post">
  <input name="text" value="{{ text }}">
  <button>Classify</button>
</form>
{% if label %}<p>Result: {{ label }}</p>{% endif %}
"""


@app.route("/", methods=["GET", "POST"])
def home():
    label = None
    text = ""
    if request.method == "POST":
        text = request.form["text"]
        label = classify(text)
    return render_template_string(PAGE, label=label, text=text)
```

What's new: you write the **route** (`/`) and the **HTML** (a Jinja template) yourself. More code than Streamlit, more control — a real web app you can style and extend. This is the Flask way.

Step 3 — FastAPI: a JSON API (and a form)

```
# app_fastapi.py   —   run with:  uvicorn app_fastapi:app --reload
from fastapi import FastAPI
from pydantic import BaseModel

from pipeline import classify
```

```
app = FastAPI()


class ClassifyRequest(BaseModel):
    text: str


@app.post("/classify")
def classify_route(body: ClassifyRequest):
    return {"label": classify(body.text)}
```

What's new: this is Doc12's FastAPI — a typed **JSON API**, not a page. A frontend (or another service) POSTs `{"text": "..."}` and gets back `{"label": "..."}`. Best when the client is code, not a person — a React app, a mobile app, another backend.

Step 4 — Django: the batteries-included framework

Django is more than one file, so here's the shape (a `view`, a `url`):

```
# views.py
from django.http import JsonResponse
import json

from pipeline import classify


def classify_view(request):
    body = json.loads(request.body)
    return JsonResponse({"label": classify(body["text"])})
```

```
# urls.py
from django.urls import path

from .views import classify_view

urlpatterns = [path("classify", classify_view)]
```

What's new: Django gives you a lot you didn't write — a user/auth system, an admin panel, a database ORM, migrations — out of the box. That's overkill for one endpoint, and exactly right when the app has logins, stored history, and an admin who manages data. Reach for it when the app is a *product*, not a demo.

You've served one function four ways

The function stayed the same; only the wrapper changed. That's the whole lesson: **the web layer is thin, and which tool you pick is about the app around it**, not the AI. Now the Core Concepts below go deeper on each, and the Practice Exercises are these exact four, on your own.

You read	Now practice
Step 1 (Streamlit)	Basic — a Streamlit UI
Step 2 (Flask)	Intermediate — a Flask app with a form
Step 3 (FastAPI)	Real-world — a FastAPI API + a form page
Step 4 (Django)	Edge — a Django view
Choosing between them	Failure/choice — pick the right tool

Core Concepts (read this first — everything you need is here)

The web-layer pattern: input → your pipeline → output

Every web app on top of an AI pipeline is the same three steps: **receive** the user's input (a form field, a JSON body), **call** your pipeline function, **return** the result (an HTML page, a JSON object). The AI is a black box the web layer calls; the frameworks differ only in how they receive and return.

How it really works

- Keep the pipeline (`classify` , `run_pipeline` , your agent) in its own module, imported by the web file. The web layer must never contain pipeline logic, and the pipeline must never import a web framework — so you can test each alone and swap the web layer without touching the AI.
- Long-running pipelines (an agent that takes 20 seconds) need a loading state (Streamlit shows a spinner; an API returns a job id and the client polls). Never let a slow call freeze the page with no feedback.
- Secrets (`OPENAI_API_KEY`) live in the server's environment, never in the page or the client — the same Doc01 rule, now with a browser that must not see your key.
- Validate the input at the web edge: an empty box, a huge paste, a wrong type. Pydantic (FastAPI) does it for you; Flask/Django/Streamlit you check by hand.

Quick cheat sheet: - Three steps: receive → call your pipeline → return. - Pipeline in one module; web layer imports it. Never mix them. - Keys stay on the server; the browser never sees them. - Give slow calls a visible loading state.

Streamlit: the fastest path to a UI

`streamlit run app.py` turns a Python script into a web page. Widgets (`st.text_input` , `st.button` , `st.file_uploader` , `st.chat_input`) are function calls; the script **re-runs top to**

bottom on every interaction, and `st.session_state` holds anything that must survive a re-run (like chat history).

How it really works - No HTML, no routes, no server code — ideal for demos, internal tools, and data apps. The cost is less control over layout and no real multi-page routing. - The re-run model surprises beginners: every click re-runs the whole file, so expensive work goes behind `@st.cache_data` / `@st.cache_resource`, and state goes in `st.session_state`. - `st.chat_message` / `st.chat_input` build a chatbot UI in a few lines — the natural front for an agent (Project 17's DeflectDesk, say).

Quick cheat sheet: - `streamlit run app.py`; widgets are function calls; the script re-runs each time. - `st.session_state` for memory; `@st.cache_*` for expensive work. - Best for demos and internal tools, not large multi-page products.

Flask: a minimal, explicit web framework

Flask gives you routes (`@app.route`) and templates (Jinja), and almost nothing else — you add what you need. You write the HTML, the form handling, and the wiring, which is more work than Streamlit but a real, extensible web app.

How it really works - A **route** maps a URL + method to a function; the function returns a response (an HTML string/template, or `jsonify(...)` for JSON). - **Jinja templates** (`render_template`) keep HTML in its own files with `{{ slots }}` — the same fixed-vs-changing split as Doc05's prompt templates, for web pages. - Flask is synchronous by default; a slow pipeline blocks a worker, so production runs it behind `gunicorn` with several workers (Doc12's deployment idea).

Quick cheat sheet: - `@app.route("/path", methods=[...])` → a function → a response. - Jinja templates for HTML; `jsonify` for JSON. - Minimal and explicit; you build up from almost nothing.

FastAPI: typed async APIs (recap from Doc12)

FastAPI, which you used in Doc12 and Project 5, is built for **JSON APIs**: typed request/response with Pydantic, automatic validation, async support, and auto-generated docs at `/docs`. It can serve HTML too, but its sweet spot is "the client is code."

How it really works - A Pydantic request model validates the body before your code runs (the "shape during" guarantee from Doc04/05, at the HTTP edge). - `async def` endpoints + `await` let one worker handle many slow calls at once — the right fit for LLM calls that spend most of their time waiting (Doc08b's event loop). - Auto docs (`/docs`) give you a tested, shareable API surface for free.

Quick cheat sheet: - Best when a frontend or another service calls you, not a person directly. - Pydantic models validate input; `async` handles slow LLM calls well. - Free interactive docs at `/docs`.

Django: batteries-included, for products

Django comes with a user/auth system, an admin panel, a database ORM, migrations, and a templating engine — a lot you'd otherwise build by hand. That's overkill for a single endpoint and exactly right when the app has accounts, stored data, and people managing it.

How it really works - The shape is `models` (database tables as Python classes) → `views` (the request handlers) → `urls` (routing) → `templates` (HTML). More structure than Flask, more included. - The **admin** (`/admin`) is the headline feature: a full CRUD UI over your models with almost no code — invaluable when non-engineers manage the app's data (saved conversations, users, feedback). - It's heavier and more opinionated; reach for it when "it's a product with users and data", not "it's a demo".

Quick cheat sheet: - models → views → urls → templates; a lot included out of the box. - The free admin panel is the reason to choose it. - For products with accounts and stored data, not quick demos.

Choosing the right tool

You need…	Use	Why
A demo or internal tool, today	**Streamlit**	A UI in pure Python, no web code
A small, custom web app you control	**Flask**	Minimal, explicit, extensible
A JSON API for a frontend or service	**FastAPI**	Typed, async, auto-docs (Doc12)
A product with users, data, an admin	**Django**	Auth, ORM, admin all included
A chatbot front for an agent	**Streamlit** (`st.chat_*`)	A chat UI in a few lines

Quick cheat sheet: - Demo → Streamlit. Small app → Flask. API → FastAPI. Product → Django. - The pipeline is identical in all four; choose by the app, not the AI.

Which framework for YOUR AI project — why, and how

For almost every AI project in this curriculum — a portfolio piece, a demo for a recruiter, an internal tool — **Streamlit is the best default**, and here's exactly why and how.

Why Streamlit wins for AI projects

- **Fastest to a working app.** Your pipeline is already a Python function; Streamlit wraps it in a UI with ~10 lines and no HTML, routes, or JavaScript. You spend your time on the AI, not the plumbing.
- **Everything an AI app needs is built in:** `st.chat_input` / `st.chat_message` for a chatbot, `st.file_uploader` for document apps (InvoiceIQ), `st.write_stream` for live LLM output, `st.spinner` for slow calls, `st.session_state` for memory.

- **Free, one-click hosting.** Push to GitHub, connect Streamlit Community Cloud, and you have a public link to share — no server to run or pay for. That link *is* your portfolio.

When to pick something else

- **FastAPI** — when the project is a **backend API** a separate frontend (React, a mobile app) or another service calls. You ship endpoints, not a UI. Pick this when "the client is code."
- **Flask** — when you want a **small custom web app** with your own HTML and routes, but none of Django's weight.
- **Django** — only when the project is a **real product**: user accounts, logins, a database of saved history, and an admin panel. Its batteries save weeks *there*, and are dead weight anywhere else.

How to ship a Streamlit project (the whole flow)

1. Keep your pipeline in its own module (`pipeline.py`) with one entry function.
2. Write `app.py` : import the function, build the UI (`st.text_input` / `st.file_uploader` → call it → `st.write` the result).
3. Add `requirements.txt` (your deps + `streamlit`) and a `.gitignore` (never commit `.env`).
4. Push to GitHub, connect Streamlit Community Cloud, set your `OPENAI_API_KEY` as a secret in its dashboard (never in the code), and deploy.
5. Share the link. For a FastAPI/Flask app instead, the same code deploys on Render, Railway, or Fly.io with a `Dockerfile` (Doc12).

The one rule again: the AI pipeline is identical whichever you pick — so choose by the **app** (a demo? an API? a product?), start with Streamlit unless a reason pushes you off it, and keep the web layer a thin wrapper.

File uploads: the key to document AI apps

Half the real AI apps take a **file**, not typed text — a PDF to extract (InvoiceIQ), a resume to match (JobMatch), a transcript to summarize. Every framework has a file input, and they all hand you the bytes the same way.

How it really works

- **Streamlit:** `uploaded = st.file_uploader("Upload a PDF", type=["pdf"])` returns a file-like object; `uploaded.read()` gives bytes, or `uploaded.getvalue()`. Save it to a temp path and pass that to your loader (Doc08's `PyPDFLoader`).
- **FastAPI:** `async def upload(file: UploadFile): data = await file.read()` — `file.filename` and the bytes. Validate the type before processing.
- **Flask:** `file = request.files["file"]; data = file.read()` — plus `<input type="file" name="file">` in a `multipart/form-data` form.

- Always validate: size limits, allowed extensions, and an empty upload. A user *will* upload a 200 MB file or the wrong type.

Quick cheat sheet: - Streamlit `st.file_uploader(type=[...])` → `.read()`; FastAPI `UploadFile` → `await .read()`; Flask `request.files["file"]`. - Save bytes to a temp file, then hand the path to your document loader. - Validate type and size — uploads are untrusted input.

Streaming the answer to the UI

A long LLM reply shouldn't appear all at once after a 15-second blank screen — it should *type out*, like ChatGPT. That's streaming, and the web layer forwards the tokens your pipeline already yields (Doc04/09 streaming).

How it really works

- **Streamlit:** `st.write_stream(token_generator)` renders tokens as they arrive. If your chain streams (`chain.stream(...)`), pass that straight in.
- **FastAPI:** return a `StreamingResponse(token_generator, media_type="text/plain")` (or Server-Sent Events) so the browser receives chunks live.
- **Flask:** return a generator from the view with `Response(gen(), mimetype="text/plain")`.
- The pipeline yields tokens; the web layer only forwards them. Don't buffer the whole answer and then "fake" streaming — forward as they come.

Quick cheat sheet: - Streamlit `st.write_stream(gen)`; FastAPI `StreamingResponse(gen, ...)`. - Your chain's `.stream()` output plugs straight in — forward, don't buffer. - Streaming is the difference between a frozen screen and a live-typing answer.

A real HTML + JavaScript frontend calling your API

"Frontend" sometimes means an actual HTML page with JavaScript — the thing a FastAPI/Flask backend serves. The page collects input and calls your API with `fetch`, then shows the result. This is the backend-plus-frontend split a production app uses.

How it really works

- The backend exposes a JSON endpoint (`POST /classify` → `{"label": ...}`), exactly the FastAPI service you built.
- The page's JavaScript calls it: `fetch("/classify", {method:"POST", headers:{"Content-Type":"application/json"}, body: JSON.stringify({text})})`, then `await res.json()` and write the result into the page.
- The browser never holds your API key — the key lives on the server, and the page only talks to *your* endpoint, which talks to the model.
- For dynamic pages without writing much JS, **HTMX** (an `hx-post` attribute on a form) is a popular middle ground: the server returns HTML fragments, no custom JavaScript.

Quick cheat sheet: - Page collects input → `fetch("/your-endpoint", ...)` → show the JSON result. - The API key stays on the server; the page only calls your endpoint. - HTMX (`hx-post`) gives dynamic pages with almost no JavaScript.

Deploying your web app

An app only counts when someone else can open it. Each tool has a standard path.

How it really works

- **Streamlit → Streamlit Community Cloud**: connect your GitHub repo, set secrets (your API key) in the dashboard, deploy. Free, a public link, zero servers.
- **FastAPI / Flask → Render, Railway, or Fly.io**, usually via a `Dockerfile` (Doc12's exact container). Set the API key as an environment variable in the host, never in the image.
- **Django** → the same hosts, plus a managed database for its ORM.
- Everywhere: secrets come from the host's environment, `requirements.txt` is pinned, and `.env` is never committed — the Doc01 / Doc12 rules, on a real server.

Quick cheat sheet: - Streamlit → Streamlit Cloud (free, one click). FastAPI/Flask → Render/Railway/Fly with a Dockerfile. - Secrets live in the host's env vars, never in the repo or image. - A deployed link is the portfolio; a localhost demo isn't shareable.

Practice Exercises

Setup: `pip install streamlit flask fastapi "uvicorn[standard]" django jinja2`. Put your code under `20_web_frontends/practice/`. Keep one `pipeline.py` with a `classify(text)` (or import a real project's pipeline) and build each UI around it.

Jump to an exercise: Streamlit · Flask · FastAPI · Django · Choice · File upload · Streaming · HTML+JS frontend

Basic — a Streamlit UI

- **What:** a one-file Streamlit app that takes text, calls `classify`, and shows the result — plus a chat version with `st.chat_input` and `st.session_state`.
- **Why:** Streamlit is the fastest way to make any project clickable; the chat version is the natural front for an agent.
- **How to code it:** `st.text_input` + `st.button` → `st.write(classify(text))`; then a chat loop that appends to `st.session_state["messages"]` and renders with `st.chat_message`.
- **Stuck?** Hint 1 · Hint 2 · Solution

Intermediate — a Flask app with a form

- **What:** a Flask app with a GET form page and a POST handler that calls `classify` and re-renders the page with the result.
- **Why:** you write the route and the HTML yourself — the real web-app pattern Streamlit hides.
- **How to code it:** one `/` route handling GET and POST; a Jinja template with a form; read `request.form["text"]` on POST. Add a `/classify` JSON route with `jsonify`.
- **Stuck?** Hint 1 · Hint 2 · Solution

Real-world — a FastAPI API + a form page

- **What:** a FastAPI service with a typed `POST /classify` JSON endpoint and a tiny HTML form at `/`, tested with `TestClient`.
- **Why:** this is the API shape a real frontend calls — and the one you already touched in Doc12, now standalone.
- **How to code it:** a Pydantic `ClassifyRequest`, a `POST /classify` returning `{"label": ...}`, and a `GET /` returning an HTML form. Test both with `fastapi.testclient.TestClient` (no server needed).
- **Stuck?** Hint 1 · Hint 2 · Solution

Edge — a Django view

- **What:** a minimal Django app — one `view`, one `url` — that serves `classify`, run against Django's test `Client`.
- **Why:** to feel Django's shape (views/urls/settings) and see why it's heavier — so you know when its batteries are worth the weight.
- **How to code it:** a `classify_view(request)` returning `JsonResponse`, a `urlpatterns` entry, and the minimal `settings` to run it. Test with `django.test.Client`.
- **Stuck?** Hint 1 · Hint 2 · Solution

Failure/choice — pick the right tool

- **What:** for five short scenarios (a hackathon demo, a public JSON API, an internal tool, a customer product with logins, a chatbot), name the right framework and one sentence why.
- **Why:** the skill isn't writing all four — it's knowing which one a real job needs, so you don't put Django on a demo or Streamlit on a product.
- **How to code it:** a `practice/framework_choice.md` with your five answers; check them against the solution's reasoning.
- **Stuck?** Hint 1 · Hint 2 · Solution

Intermediate — a file-upload app

- **What:** a Streamlit app with `st.file_uploader` that takes a `.txt` / `.pdf`, reads it, and runs a pipeline on its text — plus the same as a FastAPI `UploadFile` endpoint.

- **Why:** half the real AI apps take a file, not typed text (InvoiceIQ, JobMatch). This is the missing half of the UI.
- **How to code it:** Streamlit — `uploaded = st.file_uploader("File", type=["txt","pdf"])`, `uploaded.read()`, decode (or save to a temp path for a PDF loader), run the pipeline, show the result. FastAPI — `async def upload(file: UploadFile)`, `await file.read()`, validate the type, return the result. Test the FastAPI one with `TestClient` and a `files={...}` POST.
- **Stuck?** Hint 1 · Hint 2 · Solution

Intermediate — stream the answer

- **What:** stream an answer token-by-token to the UI — `st.write_stream` in Streamlit and a FastAPI `StreamingResponse` — so a long reply types out instead of appearing after a blank wait.
- **Why:** streaming is the difference between a frozen screen and a live, ChatGPT-style answer; it's expected of any real chat UI.
- **How to code it:** a generator that yields pieces of text (in a real app, your `chain.stream(...)`). Streamlit: `st.write_stream(gen())`. FastAPI: `return StreamingResponse(gen(), media_type="text/plain")`, tested with `TestClient` reading the streamed body.
- **Stuck?** Hint 1 · Hint 2 · Solution

Intermediate — an HTML + JavaScript frontend

- **What:** a FastAPI backend serving a real HTML page whose JavaScript `fetch`es a JSON endpoint — the backend-plus-frontend split a production app uses.
- **Why:** "frontend" sometimes means an actual page with JS calling your API; this is how a React/plain-JS client talks to your pipeline, with the key safe on the server.
- **How to code it:** a `GET /` returning an HTML page with a form and a `<script>` that `fetch("/api/classify", {method:"POST", body: JSON.stringify({text})})` and writes the result into a `<div>`; a `POST /api/classify` JSON endpoint. Test the endpoint with `TestClient`; open the page in a browser to see the fetch work.
- **Stuck?** Hint 1 · Hint 2 · Solution

Build Task — Put a UI on a Real Pipeline

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: take one real pipeline you already built — InvoiceIQ (Project 14), MeetingMind (Project 15), JobMatch (Project 16), or any agent — and give it **two** web layers: a **Streamlit** app (the demo) and a **Flask or FastAPI** app (the real service). The pipeline code is imported unchanged; only the web files are new.

Requirements: - One `pipeline.py` (or an import of the real project) with a single entry function. The web files import it and add nothing to the AI logic. - A Streamlit app that takes the input (text, or a file upload for InvoiceIQ) and shows the result. - A Flask or FastAPI app exposing the same thing as a route, with input validation (empty input → a clear message, not a crash). - A short note on which framework you'd ship and why.

Test Cases:

Scenario	Expected
Valid input through the UI	The pipeline's result is shown/returned
Empty input	A clear "please enter…" message, no crash
The same input to both UIs	The same result (the pipeline is shared)

Move On When

You can take any pipeline and put a web UI on it in the framework the job calls for — and explain, for a given app, why that framework over the other three.

Stuck? Ask for **Hint 1** or **Hint 2**. The point isn't memorizing four APIs — it's the one shared pattern (input → pipeline → output) and knowing which tool fits.